

VOID DESCENT

Documentação Técnica

Roguelike top-down procedural com WebGL

Equipe

Luis Gustavo Olímpio

Lauan Matheus

José Melquíades

João Leonardi

iCEV — Instituto de Ensino Superior
Introdução ao Desenvolvimento de Jogos

Maio de 2026

Sumário

1	Visão Geral	4
1.1	O jogo	4
1.2	Equipe	4
1.3	Stack tecnológico	4
1.4	Métricas do projeto	4
1.5	Estrutura desta documentação	5
2	Arquitetura de Componentes	6
2.1	Decomposição modular	6
2.2	Responsabilidades por módulo	6
2.3	Dependências e fluxo de controle	6
2.4	Boot sequence	8
3	Sistema de Entidades	9
3.1	Modelo orientado a objetos	9
3.2	A classe-base <code>entity_t</code>	9
3.3	Subclasses de combate	9
3.3.1	<code>entity_player_t</code>	9
3.3.2	Inimigos	11
3.4	Subclasses de projétil e efeito	11
3.5	Subclasses de pickup e cenário	11
3.6	Lifecycle e remoção	11
4	Máquina de Estados do Jogo	12
4.1	O <code>game_state</code> global	12
4.2	Catálogo de estados	12
4.3	Transições críticas	13
4.3.1	<i>Setup</i> de <i>handlers</i> por estado	13
4.3.2	Race conditions com <code>setTimeout</code>	13
4.3.3	Reset de <code>keys[]</code>	13
5	Game Loop	14
5.1	Loop Micro: lock-and-clear	14
5.2	Loop Macro: progressão da run	15
5.3	Sequência interna de um <i>frame</i>	17
6	Sistemas Procedurais	19
6.1	Geração de <i>dungeon</i>	19
6.1.1	Algoritmo de <i>room placement</i>	20
6.1.2	Validação	21
6.2	Síntese de áudio	21
6.2.1	Pipeline	22
6.2.2	Estrutura interna do sintetizador	23
6.3	Atlas de tiles e <i>tinting</i>	23

7	Build e Deploy	24
7.1	Pipeline de empacotamento	24
7.2	Comandos de <i>build</i>	25
7.2.1	Desenvolvimento local	25
7.2.2	Build do .exe para Windows	26
7.3	Cross-compile do macOS	26
7.4	Por que 73 MB?	26
7.5	Distribuição	26
7.6	Repositório	26
8	Backlog Técnico, Decisões e Aprendizados	27
8.1	Decisões técnicas	27
8.1.1	JavaScript puro sem <i>framework</i>	27
8.1.2	Atlas de tiles embarcado em <i>base64</i>	27
8.1.3	Sonant-X em vez de Web Audio API direta	27
8.1.4	Electron em vez de Tauri ou Neutralino	27
8.1.5	<i>Spatial grid</i> 32×32 com células de 16	28
8.1.6	<i>Lock-and-clear</i> via colisão lógica em vez de <i>tiles</i> físicos	28
8.2	Backlog: features descartadas ou postergadas	28
8.3	Ferramentas e <i>assets</i>	29
8.3.1	Estética visual	29
8.3.2	Áudio	29
8.3.3	Cadeia de <i>tools</i>	29
8.4	Aprendizados técnicos	29
8.4.1	<i>Race conditions</i> em <i>setTimeout</i> aninhados	29
8.4.2	Coordenadas do <i>mouse</i> requerem <i>getBoundingClientRect</i>	30
8.4.3	<i>textAlign</i> do <i>canvas</i> 2D persiste entre chamadas	30
8.4.4	<i>Lock-and-clear</i> precisa de <i>tracking</i> explícito por entidade	30
8.4.5	<i>Loop</i> de música ambiente exige alinhamento de <i>samples</i>	30
8.4.6	Cross-compile macOS para Windows funciona, com ressalvas	30
8.5	Mudanças de escopo em relação ao GDD	30
8.5.1	Divergências de <i>gameplay</i>	30
8.5.2	Divergências de <i>interface</i>	31
8.5.3	Divergências de <i>geração</i>	32
8.5.4	Adições não previstas no GDD	32
8.6	Cronograma: planejado vs executado	33
8.7	Uso de IA como ferramenta de apoio	33
8.7.1	Abstrações matemáticas pontuais	33
8.7.2	Pesquisa de bibliotecas e <i>trade-offs</i>	33
8.7.3	Revisão de código	34
8.7.4	Polimento da documentação	34
8.7.5	O que a IA não fez	34
9	Considerações Finais	35
9.1	O que foi entregue	35
9.2	O que aprendemos	35
9.3	O que faríamos diferente	35
9.4	O que vem depois	36
9.5	Encerramento	36

Lista de Figuras

2.1	Diagrama de Componentes, com os módulos do Void Descent e suas dependências	7
3.1	Diagrama de Classes, com a hierarquia de <code>entity_t</code>	10
4.1	Diagrama de Estados do <code>game_state</code>	12
5.1	Diagrama de Atividade do ciclo de combate por sala	15
5.2	Diagrama de Atividade da progressão completa da <i>run</i>	16
5.3	Diagrama de Sequência da execução interna de <code>game_tick</code> no estado <code>playing</code>	18
6.1	Diagrama de Sequência da geração procedural de um andar	20
6.2	Diagrama de Sequência do <i>pipeline</i> de síntese de áudio	22
7.1	Diagrama de Deployment do <i>pipeline</i> de <i>build</i> e <i>runtime</i>	25

1. Visão Geral

1.1 O jogo

Void Descent é um *roguelike* top-down em tempo real construído em JavaScript puro com renderização WebGL. Cada partida regenera todo o conteúdo do jogo (mapas, atlas de tiles, áudio, comportamento de inimigos) por algoritmos em tempo de execução, sem nenhum *asset* pré-fabricado.

A premissa é descer três andares procedurais, *Antecâmara*, *Subsistemas* e *Núcleo*, eliminando inimigos sala por sala e coletando armas e *upgrades*, até derrotar o boss final. A morte é permanente; cada *run* é uma nova execução do gerador.

1.2 Equipe

- Luis Gustavo Olímpio
- Lauan Matheus
- José Melquíades
- João Leonardi

1.3 Stack tecnológico

- **JavaScript ES6+**, sem *frameworks*, ESLint ou bundlers. Carregamento direto de `<script>` em ordem.
- **WebGL 1.0** com *shaders* de vértice e fragmento personalizados, e iluminação por vértice de até 32 luzes pontuais simultâneas.
- **Sonant-X**, *port* JavaScript do sintetizador Sonant (Marcus Geelnard, Nicolas Vanhoren), gera todo o áudio em tempo de *boot*.
- **Web Audio API**: `AudioContext` e `GainNode` mestre para *routing* unificado e controle de *mute*.
- **Electron 32** para empacotamento como `.exe portable` Windows x64, via `electron-builder`.

1.4 Métricas do projeto

Métrica	Valor
Linhas de código (JS+HTML)	~2.100
Tamanho do código fonte	77 KB
Quantidade de arquivos JS	8
Tamanho do <code>.exe portable</code>	73 MB
Chromium embutido	~60 MB
Node runtime	~10 MB
Bundle do jogo (asar)	~80 KB
Tempo médio de <i>run</i>	8 a 12 minutos

O contraste entre o código fonte (77 KB) e o `.exe` final (73 MB) decorre da escolha de empacotamento via Electron, conforme especificação do GDD para garantir experiência *nativa* de *desktop*. A geração procedural reside na camada do *bundle*, não no *wrapper* de distribuição.

1.5 Estrutura desta documentação

A documentação está dividida em nove capítulos, cada um focando uma dimensão ortogonal do projeto:

1. **Visão Geral**, este capítulo.
2. **Arquitetura de Componentes**, com a divisão modular e as dependências (*component diagram*).
3. **Sistema de Entidades**, com a hierarquia de classes do mundo do jogo (*class diagram*).
4. **Máquina de Estados**, com as transições do estado global (*state diagram*).
5. **Game Loop** *micro* (sala-a-sala) e *macro* (run completa), representado por *activity diagrams*.
6. **Sistemas Procedurais**: geração de *dungeon*, síntese de áudio e *tinting* de atlas (*sequence diagrams*).
7. **Build e Deploy**, com o *pipeline* de empacotamento e distribuição (*deployment diagram*).
8. **Backlog Técnico, Decisões e Aprendizados**, com o caminho real do projeto: o que escolhemos, o que descartamos e o que aprendemos.
9. **Considerações Finais**, com o balanço do que foi entregue, o que faríamos diferente e o que vem depois.

2. Arquitetura de Componentes

2.1 Decomposição modular

O código está dividido em oito módulos JavaScript, cada um com responsabilidade única e bem delimitada. O **Component Diagram** é o tipo UML adequado para representar a divisão de *software* em unidades *deployáveis* e suas dependências de uso.

2.2 Responsabilidades por módulo

src/vd/random.js Gerador de números pseudoaleatórios determinístico, baseado em *Linear Congruential Generator* (LCG) de 32 bits. É *seedado* explicitamente a cada andar (`random_seed(0xBADC0DE1 + floor_id)`), o que garante reprodutibilidade do mapa para uma dada combinação de *seed* e andar.

src/vd/sonantx.js *Port* JavaScript do sintetizador Sonant. Implementa osciladores (sin, quadrada, dente-de-serra, triangular), envelopes ADSR, filtros (lopass, hipass, bandpass, notch), LFO e *delay*, todos rodando sobre `Float32Array`.

src/vd/audio.js Define os *patches* de instrumentos (oito SFX e a faixa ambiente). Inicializa o `AudioContext`, cria o `GainNode` mestre (`audio_master_gain`) que centraliza o controle de *mute*, e coordena a geração assíncrona dos `AudioBuffers` no *boot*.

src/vd/renderer.js *Pipeline* WebGL. Compila os *shaders* de vértice e fragmento, gerencia o atlas de tiles (`renderer_bind_image`), expõe primitivas de geometria (`push_quad`, `push_floor`, `push_block`, `push_sprite`) e gerencia até 32 luzes pontuais por *frame*.

src/vd/dungeon.js Algoritmo de *room placement* com validação posterior. Posiciona retângulos não sobrepostos, conecta-os por corredores em forma de L, marca tipos especiais (`start`, `chest`, `boss`) e converte o resultado em geometria via chamadas a `push_block`/`push_floor`.

src/vd/entities.js Hierarquia de classes do mundo simulado: classe-base `entity_t`, *player*, quatro tipos de inimigos, *boss*, projéteis, partículas, explosões e *pickups*. Detalhada no Capítulo 3.

src/vd/terminal.js *Overlay* HUD textual no estilo terminal de boot. Renderiza no elemento DOM `<code id="a">` (fora do *canvas*) e exibe avisos temporários como “ANDAR 1 :: ANTECAMARA” ou “PORTAS TRANCADAS”.

src/vd/game.js Módulo orquestrador. Mantém o `game_state` global, implementa o `game_tick` (despachado por estado), e gerencia a *shop* de *upgrades*, o *lock-and-clear* de salas, o *spatial grid* de colisão e a HUD principal.

2.3 Dependências e fluxo de controle

A arquitetura segue um padrão estritamente **em camadas**, sem ciclos:

1. **Camada de utilitários puros**, sem dependências: `random.js`, `sonantx.js`.
2. **Camada de infraestrutura**: `renderer.js`, `audio.js` (que depende de `sonantx.js`), `terminal.js`.

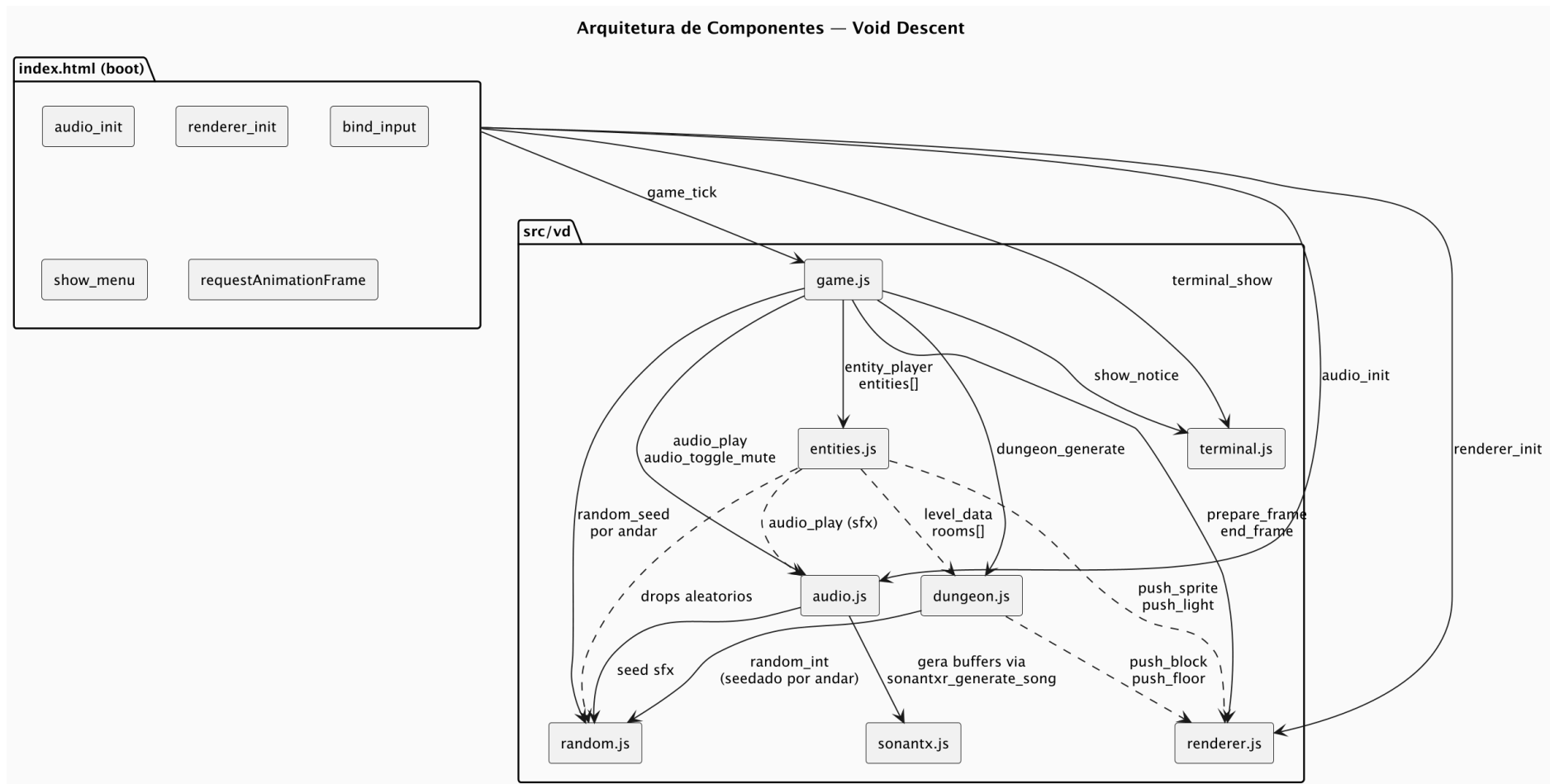


Figura 2.1: Diagrama de Componentes, com os módulos do Void Descent e suas dependências

3. **Camada de geração:** `dungeon.js`, que depende de `random.js` e da API de geometria de `renderer.js`.
4. **Camada de simulação:** `entities.js`, que depende das três anteriores via globais.
5. **Orquestração:** `game.js`, que depende de tudo.

A ausência de um sistema de módulos (CommonJS, ESM) é intencional. Todos os símbolos vivem no escopo global definido pela ordem de inclusão dos `<script>` em `index.html`. O padrão minimalista é coerente com a proposta procedural do projeto e elimina o custo de *boot* de um *bundler*.

2.4 Boot sequence

A inicialização do programa, definida em `index.html`, segue uma ordem rígida:

1. `audio_init()` cria o `AudioContext` e dispara a síntese assíncrona da música e dos SFX.
2. `renderer_init()` cria o contexto WebGL, compila os *shaders* e aloca o *vertex buffer*.
3. `bind_input()` registra os *handlers* de teclado e mouse no `document`.
4. `show_menu()` define `game_state = 'menu'` e instala os *handlers* específicos da tela de título.
5. `requestAnimationFrame(game_tick)` inicia o *loop* principal.

3. Sistema de Entidades

3.1 Modelo orientado a objetos

Toda forma de vida (e morte) no mundo do Void Descent é uma `entity_t`, uma classe-base ECMAScript 2015 com herança simples. O **Class Diagram** comunica simultaneamente herança, atributos, métodos e relacionamentos.

3.2 A classe-base `entity_t`

Todas as entidades compartilham:

- **Cinemática:** posição (`x`, `y`, `z`), velocidade (`vx`, `vy`, `vz`), aceleração (`ax`, `ay`, `az`) e atrito (`f`).
- **Visualização:** `s` (índice do tile no atlas) e `_dead` (*flag* de remoção).
- **Saúde:** `h` (*hit points*), inicializado a 5 e sobrescrevível no `_init`.

A interface comportamental é definida por seis métodos sobrescrevíveis:

`_init(param)` *Hook* de inicialização chamado pelo construtor. Permite à subclasse personalizar atributos sem reimplementar a cadeia de inicialização.

`_update()` Avança a física por `time_elapsed` segundos. A implementação base aplica forças, integra a posição e testa colisão com paredes via `_collides`.

`_collides(x, z)` Checa se a posição alvo está dentro de um tile sólido. Sobrescrito em `entity_player_t` para implementar a barreira do *lock-and-clear*.

`_check(other)` Chamado para cada par de entidades cuja AABB se intersecta. Implementa *pickup*, dano por contato, separação entre inimigos, entre outros.

`_render()` Emite primitivas para o *renderer* (`push_sprite`, `push_light`).

`_kill()` Marca a entidade como `_dead`, agendando sua remoção no fim do *frame*.

3.3 Subclasses de combate

3.3.1 `entity_player_t`

O jogador é único em vários aspectos:

- Sobrescreve `_collides` para impedir saída de salas trancadas.
- Mantém estado de *combo* (`combo`, `combo_timer`) e pontuação.
- Gerencia o *cooldown* do *dash* (`_dash_cooldown`, `_dashing`) e da arma equipada (`_last_shot`).
- Em `_update`, calcula o ângulo de mira a partir da posição do mouse e do *offset* da câmera, e despacha o ataque pelo *type* da arma (*melee* ou *ranged*).

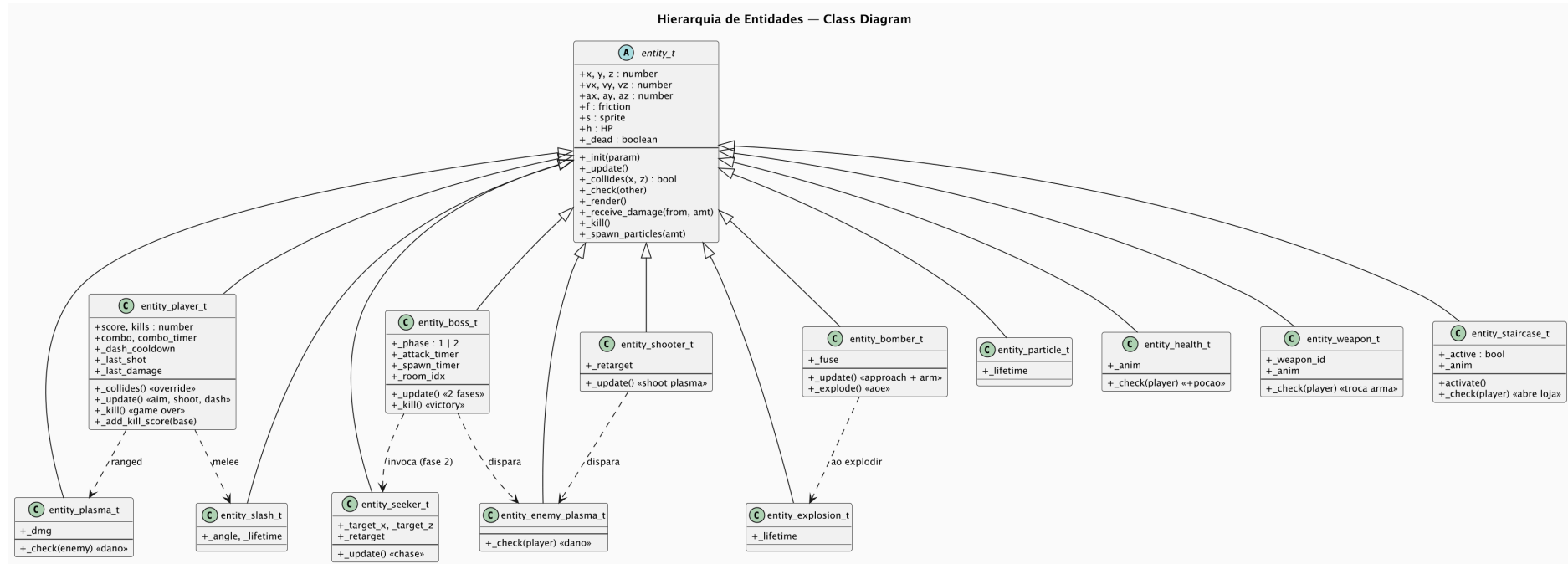


Figura 3.1: Diagrama de Classes, com a hierarquia de `entity_t`

3.3.2 Inimigos

entity_seeker_t (HP 3). Persegue o jogador em linha reta com *retarget* periódico. Comportamento mais simples, introduzido no Andar 1.

entity_shooter_t (HP 5). Mantém distância de 24 unidades do jogador, e dispara em cone duas **entity_enemy_plasma_t** ao detectá-lo. Aparece a partir do Andar 2.

entity_bomber_t (HP 2). Persegue agressivamente; ao chegar a 20 unidades de distância, ativa um *fuse* de 0,8 s e explode, causando dano em raio de 28 unidades. Introduzido no Andar 3.

entity_boss_t (HP 70). Adversário do Andar 3, com duas fases:

1. Fase 1 (HP > 35): atira anéis de 7 plasmas em padrão rotativo.
2. Fase 2 (HP ≤ 35): abandona o padrão rotativo, avança em *melee* e invoca grupos de 3 **entity_seeker_t**.

3.4 Subclasses de projétil e efeito

entity_plasma_t Projétil do jogador (*ranged*). Move-se em linha reta, hospeda *_dmg* configurável pela arma, e é destruído ao colidir com parede ou inimigo.

entity_enemy_plasma_t Versão adversária, que só causa dano ao jogador.

entity_explosion_t Efeito visual transitório de 1 segundo, com luz pulsante para *feedback* de impacto.

entity_slash_t *Burst* de partículas com *flash* de luz para o golpe *melee*. *Lifetime* curto, de 180 ms.

entity_particle_t Partícula com gravidade e *bounce*, gerada por impactos.

3.5 Subclasses de pickup e cenário

entity_health_t *Pickup* de poção (curativo de 1 HP). Adiciona ao *slot* de consumível do jogador (capacidade 1). Spawn aleatório, com 50% de chance por sala regular, ou garantido na sala do baú.

entity_weapon_t *Pickup* de arma. Sorteia um índice diferente da arma atualmente equipada e substitui ao toque.

entity_staircase_t Escada de descida. Inicialmente inativa (*_active* = *false*), é ativada por *count_enemies* quando todos os inimigos da sala estão mortos. Ao ser tocada, dispara *show_shop* (em andares não-finais) ou termina o jogo (no último andar).

3.6 Lifecycle e remoção

A remoção de entidades segue um *deferred deletion* para preservar a estabilidade da iteração:

1. *_kill()* marca *_dead* = *true* e empurra a referência em *entities_to_kill*.
2. Iterações subsequentes do *_update*, *_check* e *_render* no mesmo *frame* testam *_dead* e pulam a entidade.
3. No fim do *frame*, *entities* = *entities.filter(...)* reconstrói o *array* sem as entidades marcadas.

4. Máquina de Estados do Jogo

4.1 O `game_state` global

Uma única variável `game_state` controla o comportamento do programa, assumindo nove valores discretos. O `game_tick` despacha lógica distinta para cada estado; transições são disparadas por entrada do usuário, eventos da simulação ou `setTimeouts` explícitos.

O **State Machine Diagram**, também conhecido como *statechart*, comunica simultaneamente os estados, transições e condições de guarda.

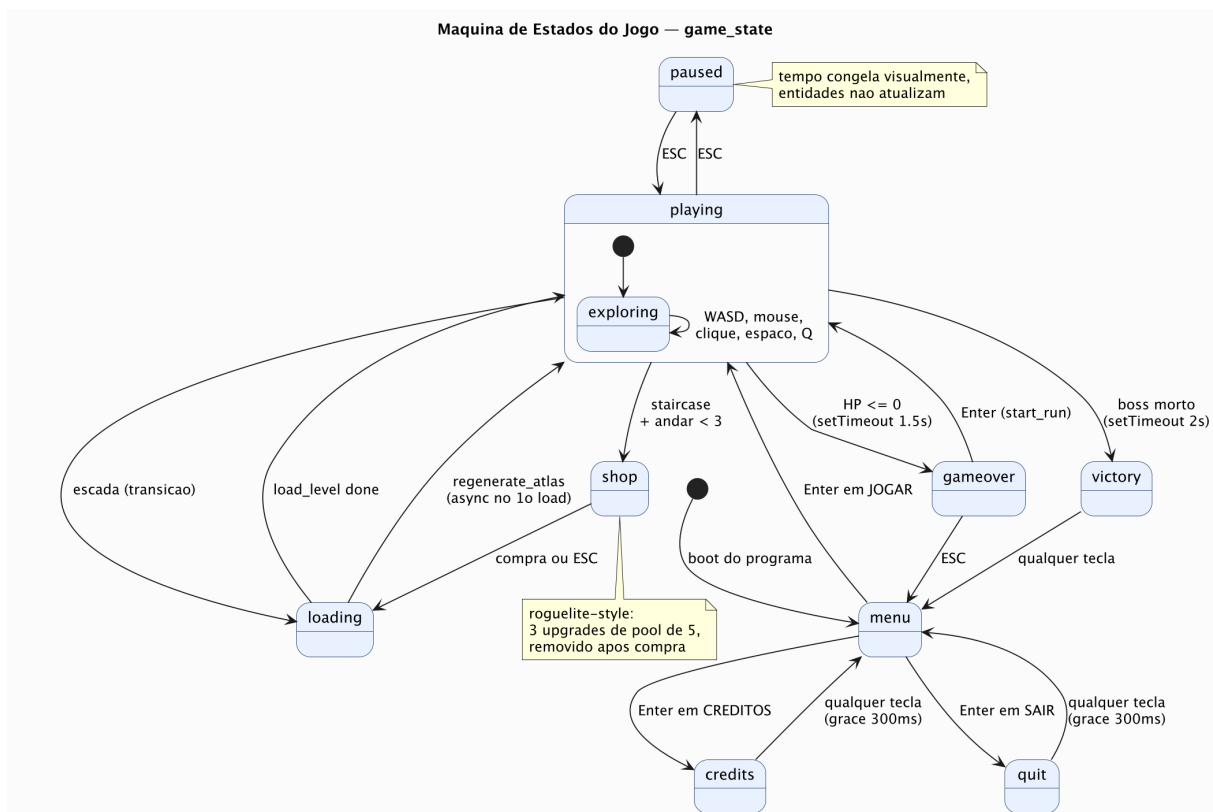


Figura 4.1: Diagrama de Estados do `game_state`

4.2 Catálogo de estados

menu Tela de título. Exibe três opções (JOGAR, CRÉDITOS, SAIR) navegáveis por setas ou WS, com confirmação por Enter ou Espaço. É o estado de entrada após o *boot* e ponto de retorno garantido a partir de qualquer estado terminal.

credits Tela de créditos com a equipe e referências de *stack*. Implementa um *grace period* de 300 ms para ignorar o *keyup* da própria tecla que abriu a tela. Qualquer tecla ou clique após esse intervalo retorna a **menu**.

quit Tela de despedida “SISTEMA DESLIGADO”. Tenta `window.close()`, que funciona no contexto Electron e falha silenciosamente no *browser*. O *fallback* para **menu** acontece via tecla ou clique.

loading Estado transitório enquanto `regenerate_atlas` aguarda o `Image` carregar. Acontece apenas no primeiro andar de cada sessão, pois chamadas subsequentes são síncronas com a imagem em cache.

playing Estado central. O `game_tick` executa as três fases do *frame* (`update` → `collision` → `render`), atualiza a HUD e checa o *lock-and-clear*.

paused Pausa por ESC. Sobrescreve os *handlers* de teclado para ignorar tudo exceto novo ESC. O `game_tick` faz *early return* sem processar entidades nem renderizar; o `canvas` mantém o último *frame* desenhado, e o `hud_ctx` recebe uma camada semi-transparente com o texto “PAUSADO”.

shop Loja entre andares, ativada quando o jogador toca a escada em um andar não-final. Apresenta três *upgrades* sorteados de um *pool* persistente de cinco. *Upgrades* comprados são removidos do *pool* para o resto da *run*.

gameover Tela de derrota. Disparada pelo `setTimeout` de 1500 ms agendado em `entity_player_t._kill`. Suporta dois caminhos: ENTER reinicia a *run*, e ESC retorna ao menu.

victory Tela de vitória. Disparada pelo `setTimeout` de 2000 ms em `entity_boss_t._kill`. Mostra estatísticas finais (tempo, kills, score, arma) e retorna ao menu por qualquer entrada.

4.3 Transições críticas

4.3.1 Setup de *handlers* por estado

Cada estado instala seu próprio conjunto de *handlers*: `onkeydown`, `onkeyup`, `onmousedown` e `onmouseup`. A função `bind_input` restaura os *handlers* de *gameplay* ao entrar em `playing`. Outros estados usam o `_bind_dismiss`, um *helper* que adiciona o *grace period* e responde em `keyup` ou `mouseup`, ou implementam *handlers* ad-hoc.

4.3.2 Race conditions com `setTimeout`

A morte do jogador agenda `show_gameover_screen` em 1500 ms, e a morte do *boss* agenda `game_victory` em 2000 ms. Se as duas mortes acontecerem no mesmo *frame* (cenário plausível, em que a explosão final do *boss* mata o jogador), os dois `setTimeouts` ficam pendentes e disparam em janelas diferentes. As transições estão protegidas por guardas idempotentes:

- `show_gameover_screen` ignora se `game_state` já é `'gameover'` ou `'victory'`.
- `game_victory` ignora se `game_state` já é `'victory'`.

A vitória, porém, *sobrescreve* a derrota. A interpretação de design é direta: se o jogador matou o *boss*, o objetivo foi cumprido, independentemente de uma morte contemporânea.

4.3.3 Reset de `keys[]`

Toda transição que entra em `playing` (a partir de `loading`, `shop` ou `gameover`) chama `for (var k in keys) keys[k] = 0`; antes de devolver o controle. A linha evita o problema de *stuck keys*, em que uma tecla pressionada durante o estado anterior ficaria registrada como ativa quando o jogo retomasse.

5. Game Loop

O *game loop* do Void Descent opera em dois níveis: **micro** (dentro de uma sala, segundo a segundo) e **macro** (ao longo de uma *run*, minuto a minuto). **Activity Diagrams** representam fluxos com decisões e iteração, e são o tipo UML adequado para ambos.

5.1 Loop Micro: lock-and-clear

O *loop* micro é o ciclo *lock-and-clear* clássico de roguelikes. Ao entrar em uma sala com inimigos, as “portas” trancam, implementadas como restrição lógica de movimento do jogador, não como tiles físicos. O combate continua até que todos os inimigos *originalmente associados àquela sala* sejam eliminados. A associação é feita no *spawn* via `e._room_idx = i`, evitando o *exploit* clássico em que um inimigo escapando para o corredor liberaria a sala falsamente.

A duração típica de um ciclo micro é de 5 a 10 segundos. As ações disponíveis ao jogador acontecem em paralelo:

- **Movimento:** WASD, ~128 unidades por segundo, escalado por `player_speed_mult`.
- **Ataque:** o clique esquerdo dispara projétil (*ranged*) ou aplica dano em arco (*melee*).
- **Dash:** ESPAÇO ativa 0,2s de invencibilidade com aceleração de 400 unidades por segundo, com *cooldown* de 1,5s.
- **Cura:** Q consome 1 poção e restaura 1 HP, se houver *slot* cheio e o jogador estiver abaixo do HP máximo.

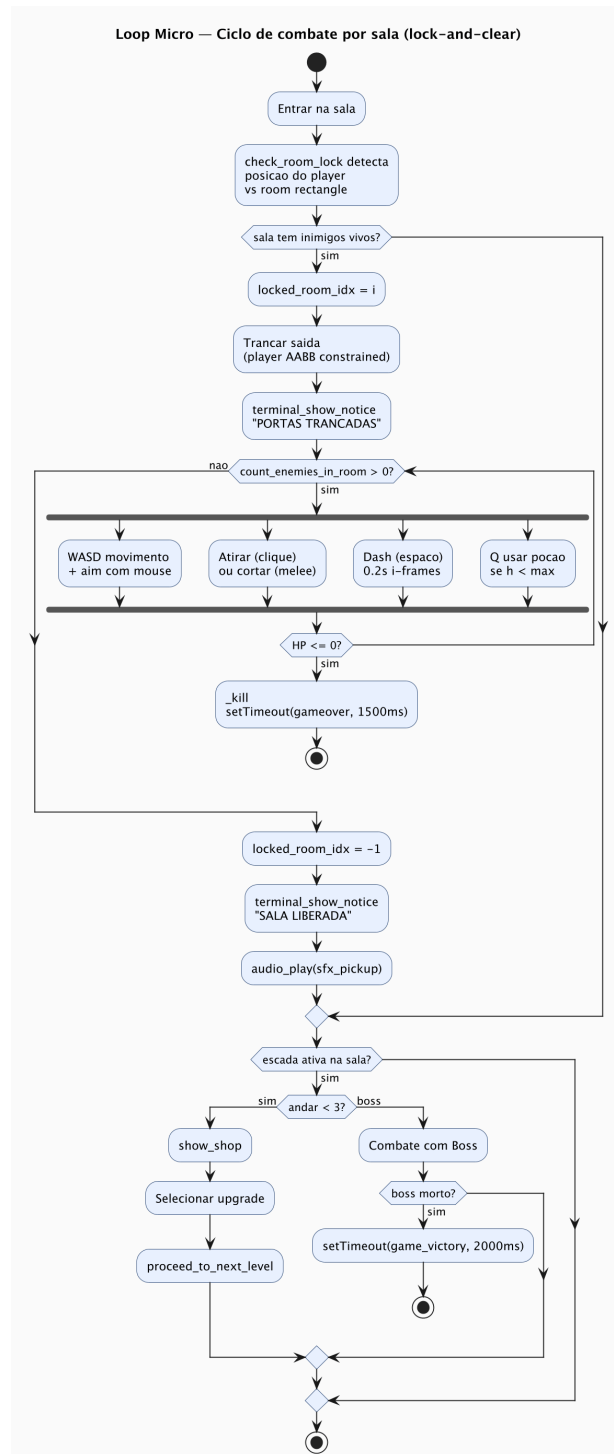


Figura 5.1: Diagrama de Atividade do ciclo de combate por sala

5.2 Loop Macro: progressão da run

O *loop* macro segue a curva de dificuldade de três atos prescrita no GDD:

1. **Andar 1, Antecâmara** (paleta ciano). Apresentação suave: apenas **Seekers**, salas espaçosas, drops generosos. Ensino implícito da mecânica básica de movimentar-se e atacar.
2. **Andar 2, Subistemas** (paleta amarela). Introdução dos **Shooters**. Pela primeira vez, o jogador precisa usar o *dash* ofensivamente para desviar de projéteis. A complexidade

tática aumenta, sem mudança fundamental nas ferramentas.

3. **Andar 3, Núcleo** (paleta magenta). Clímax: **Bombers** introduzidos, arena curta, culminando no boss em duas fases. A brevidade é proposital, pois após a extensão do Andar 2 o ritmo acelera deliberadamente em direção ao confronto final.

Entre andares (após Andar 1 e Andar 2), o jogador acessa a *shop* de *upgrades*. O *pool* de cinco *upgrades* se esvazia ao longo da *run*; combinado ao *permadeath*, isso produz escolhas únicas que não se repetem.

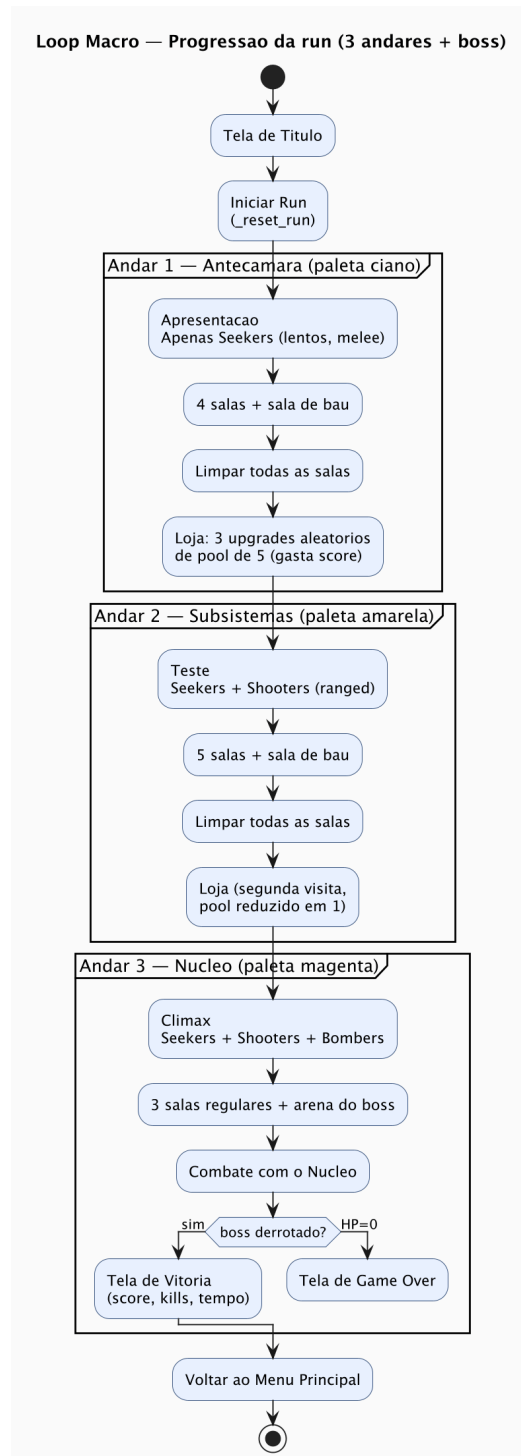


Figura 5.2: Diagrama de Atividade da progressão completa da *run*

5.3 Sequência interna de um *frame*

A função `game_tick`, invocada por `requestAnimationFrame` a ~ 60 Hz, executa três fases estritamente ordenadas:

1. **Update:** para cada entidade viva, `_update` atualiza posição e estado interno. A colisão com paredes acontece dentro de `_update`, via `_collides`. Esta fase é puramente local, sem interação entre entidades.
2. **Collision:** `check_collisions_grid` executa o *broad phase* via *spatial hash*. Cada entidade é *bucketizada* numa célula de 16×16 unidades. Para cada par de entidades em células vizinhas (3×3 *neighborhood*), o teste AABB é executado e, em caso de interseção, `e1._check(e2)` e `e2._check(e1)` são invocados. A deduplicação de pares é feita por um campo temporário `_gid`, atribuído pelo índice no *array* durante a construção do *grid*.
3. **Render:** cada entidade emite suas primitivas via `push_sprite` e `push_light`. Após o *loop*, `renderer_end_frame` executa o *draw call* único do WebGL.

A separação em fases foi introduzida durante a refatoração do *spatial grid*. O modelo anterior, um *loop* aninhado $O(n^2)$ que intercalava `_update`, `_check` e `_render` em cada iteração, não escalava para os ~ 400 inimigos por *run* do modo de dificuldade alta.

Após as três fases, a HUD é desenhada no `hud_ctx` (*canvas* 2D separado), `check_room_lock` verifica transições de sala, e o *deferred deletion* remove as entidades marcadas mortas.

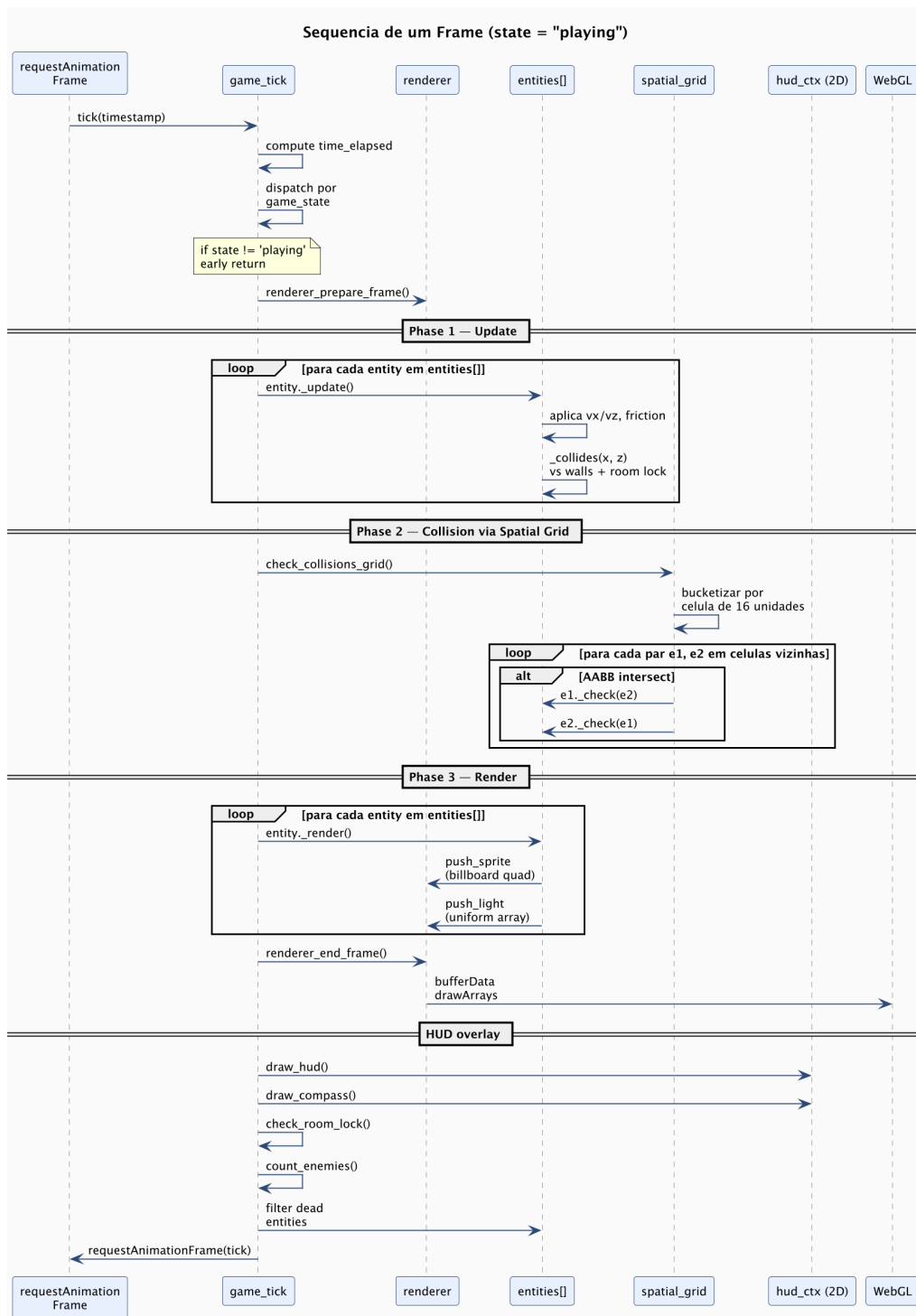


Figura 5.3: Diagrama de Sequência da execução interna de `game_tick` no estado `playing`

6. Sistemas Procedurais

A premissa fundamental do Void Descent é que *nenhum asset é importado*. Texturas, geometria, áudio, comportamento de inimigos e mapas são gerados algoritmicamente a cada *run*. Este capítulo descreve os três sistemas procedurais centrais usando **Sequence Diagrams**, que representam interação ordenada entre componentes ao longo do tempo, o tipo UML adequado para descrever *processos* multi-etapas.

6.1 Geração de *dungeon*

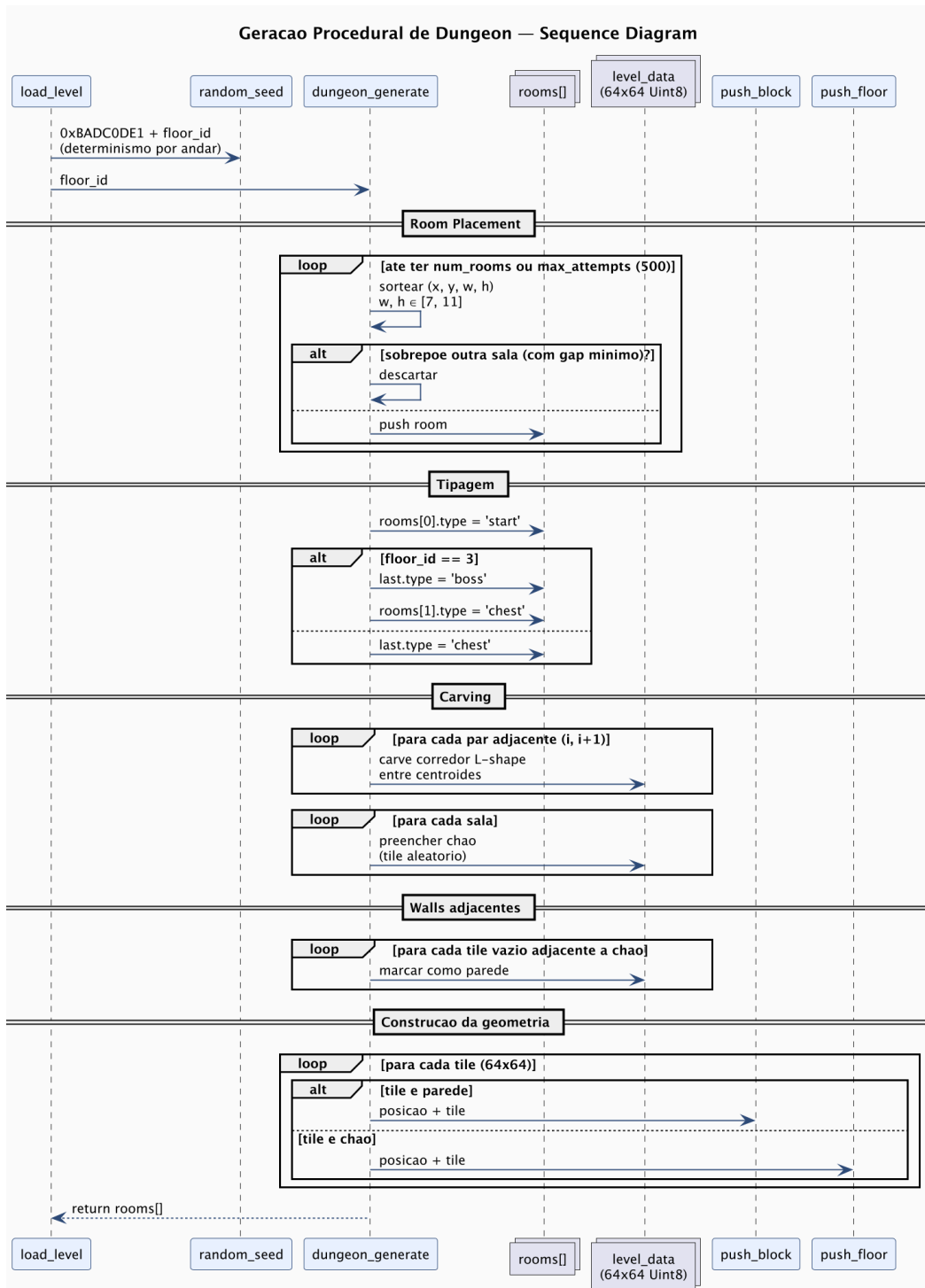


Figura 6.1: Diagrama de Sequência da geração procedural de um andar

6.1.1 Algoritmo de *room placement*

A função `dungeon_generate(floor_id)` segue cinco etapas:

1. **Seeding.** `random_seed(0xBADC0DE1 + floor_id)` reinicia o LCG. Andares com mesmo *seed* produzem o mesmo mapa, garantindo determinismo do *layout* dentro de uma *run*. Isso é importante para que `count_enemies_in_room` identifique consistentemente a mesma sala em *frames* diferentes.

2. **Posicionamento de salas.** Em um *loop* com até 500 tentativas, são sorteados retângulos com $w, h \in [7, 11]$ tiles e x, y válidos no mapa 64×64 . Cada candidato é rejeitado se sobrepõe (com *gap* mínimo de 1 tile) qualquer sala já aceita. O número alvo é $4 + \text{floor_id}$ salas, exceto no Andar 3, que tem 5.
3. **Tipagem.** `rooms[0]` é a sala inicial; a última é **chest** (Andar 1 e 2) ou **boss** (Andar 3). No Andar 3 a segunda sala também é **chest**, garantindo equipamento antes do confronto final.
4. **Carving** dos corredores. Para cada par de salas adjacentes na lista, um corredor em forma de L é escavado conectando os centroides, com escolha aleatória da ordem (horizontal-primeiro ou vertical-primeiro).
5. **Construção da geometria.** Cada tile é convertido em chamada a `push_floor` (chão) ou `push_block` (parede). O resultado é *baked* no *vertex buffer* do WebGL. Não há atualização incremental durante o *gameplay*, o que permite que `level_num_verts` marque o fim da geometria estática.

6.1.2 Validação

O posicionamento por descarte tem chance teórica de gerar mapas onde uma sala fique ilhada. Na prática, com 500 tentativas e o algoritmo de conexão sequencial $i \rightarrow i+1$, todas as salas ficam alcançáveis a partir da inicial via grafo linear de corredores.

6.2 Síntese de áudio

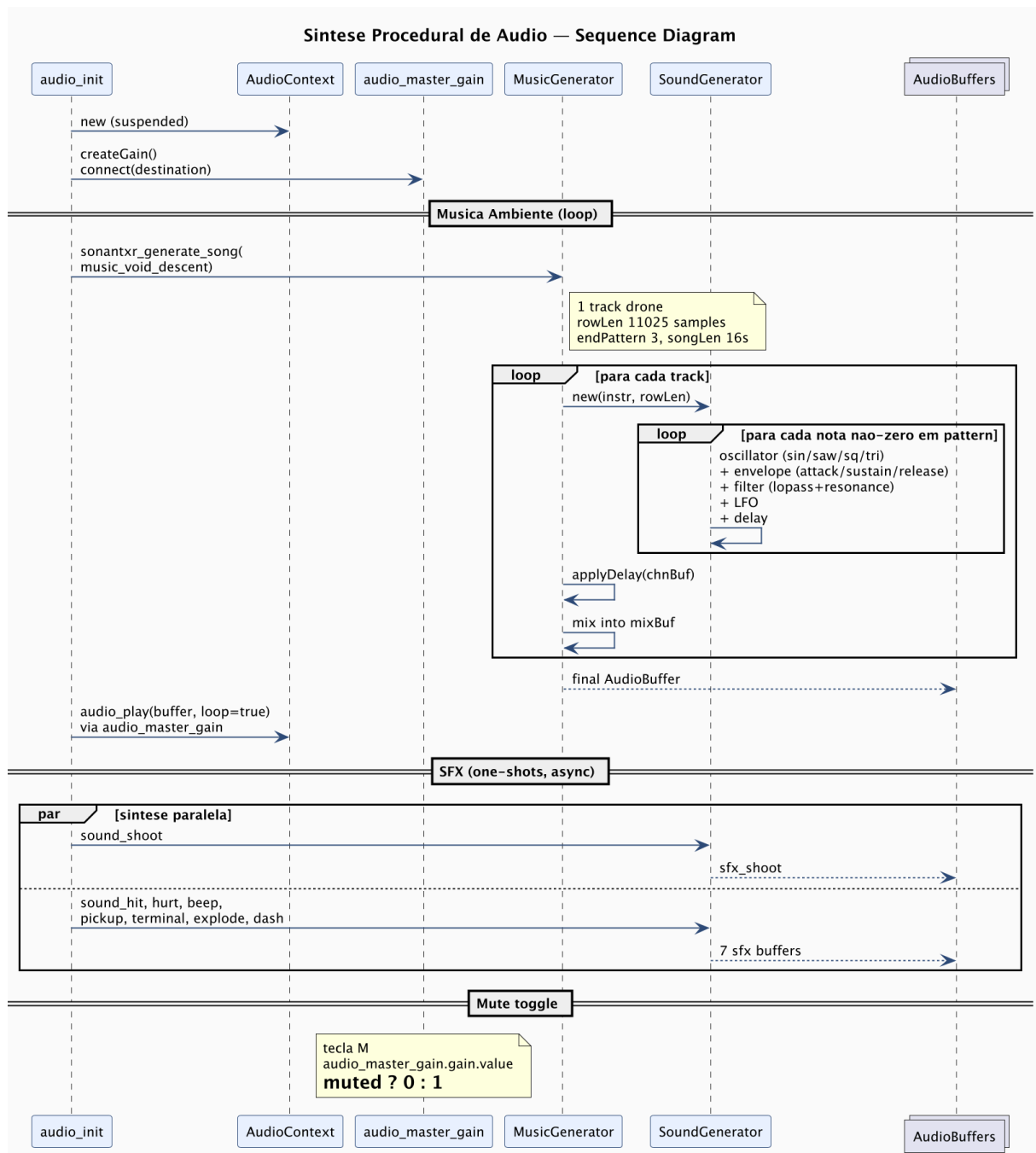


Figura 6.2: Diagrama de Sequência do *pipeline* de síntese de áudio

6.2.1 Pipeline

A `audio_init` é assíncrona:

1. Cria o `AudioContext`, inicialmente *suspended* pela política de *autoplay*, e resumido na primeira interação do usuário.
2. Cria `audio_master_gain` (um `GainNode`) e o conecta ao `destination`. Todas as fontes posteriores passam por este nó, permitindo *mute* unificado via `audio_master_gain.gain.value = 0`.
3. Dispara a síntese assíncrona da música. `sonantxr_generate_song` processa cada *track*

em *chunks* de 33 ms, intercalando com `setTimeout(0)` para não bloquear a *main thread* excessivamente.

4. Em paralelo, dispara a síntese de oito SFX (*shoot, hit, hurt, beep, pickup, terminal, explode, dash*). Cada um é síncrono, mas curto.

6.2.2 Estrutura interna do sintetizador

Cada nota processada em `SoundGenerator._genSound` executa as seguintes etapas, sobre os $\sim 10\,000$ *samples* de duração da nota:

- Dois osciladores (`osc1, osc2`) com formas selecionáveis (*sin, square, saw, tri*) e *detune* aplicável.
- Envelope ADSR (`attack, sustain, release`) calculado por *sample*.
- Filtro biquad de estado configurável (*lopass, hipass, bandpass, notch*) com ressonância.
- LFO modulando a frequência do filtro.
- *Pan* e *delay* aplicados em pós-processamento sobre o `Float32Array` resultante.

A faixa ambiente atual usa um único *track* de *drone* com `rowLen = 11025` e `songLen = 16s`, gerando um *loop* sem cortes audíveis.

6.3 Atlas de tiles e *tinting*

O atlas de tiles é embarcado no código como *string* `base64` (`TILES_B64` em `game.js`). No *boot* de cada andar, `tint_atlas` aplica um *hue shift* ao atlas inteiro pixel-a-pixel, produzindo a paleta característica de cada andar: ciano para o Andar 1 ($\Delta H = 0,55$), laranja para o Andar 2 ($\Delta H = 0,15$) e magenta para o Andar 3 ($\Delta H = 0,75$). O resultado é uma textura `tile_atlas_size2` vinculada ao GL via `renderer_bind_image`.

A conversão de espaço é `RGB` $\rightarrow$ `HSL` $\rightarrow$ `shifted HSL` $\rightarrow$ `RGB`, com saturação amplificada em 40% para preservar o *look neon* sobre fundo preto.

7. Build e Deploy

7.1 Pipeline de empacotamento

O Void Descent é distribuído como executável `.exe` portátil para Windows x64, empacotado via Electron e `electron-builder`. Para representar a topologia de *nodos* físicos e lógicos, e os *artifacts* que circulam entre eles, o tipo de diagrama UML adequado é o **Deployment Diagram**.

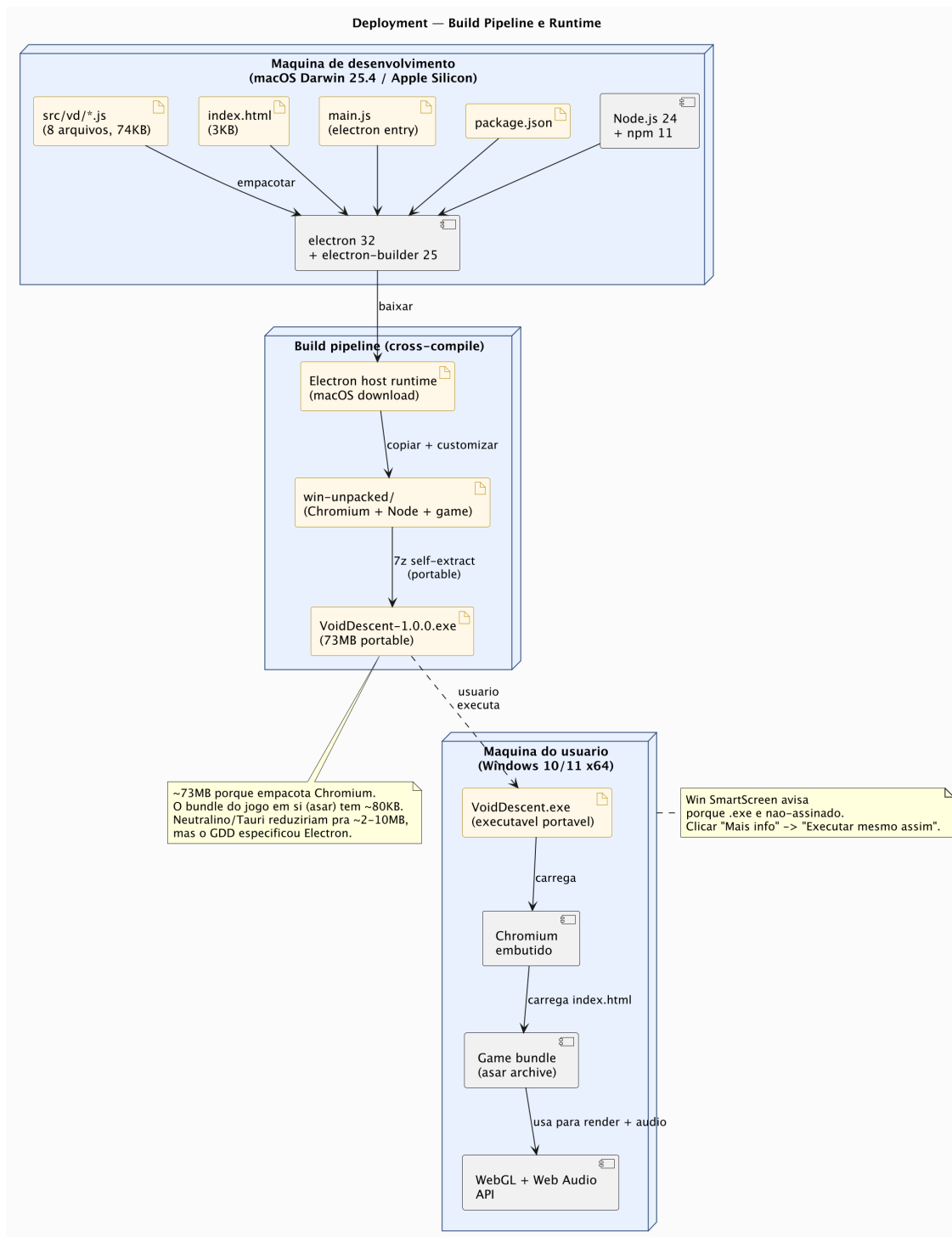


Figura 7.1: Diagrama de Deployment do *pipeline* de *build* e *runtime*

7.2 Comandos de *build*

7.2.1 Desenvolvimento local

Para rodar a versão de desenvolvimento, com *hot reload* manual via *refresh* do Electron:

```
npm install
npm start
```

7.2.2 Build do .exe para Windows

A configuração em `package.json` define o *target portable*, um executável auto-extraível sem instalador:

```
npm run build:win
```

A saída é `dist/VoidDescent-1.0.0.exe`, com ~73 MB. Em paralelo, `electron-builder` produz `dist/win-unpacked/` (a árvore de arquivos antes do empacotamento, útil para *debug*) e arquivos auxiliares (`builder-debug.yml`, `blockmaps`) que são ignorados pelo `.gitignore`.

7.3 Cross-compile do macOS

A geração do .exe para Windows acontece no *host* macOS, sem necessidade de Wine ou máquina virtual Windows. O *target portable* é uma das poucas configurações que o `electron-builder` cross-compila nativamente; alvos NSIS (instalador) ou MSI exigem Wine no PATH.

7.4 Por que 73 MB?

A composição aproximada do binário .exe é a seguinte:

Componente	Tamanho
Chromium (V8 e Blink)	~60 MB
Node.js <i>runtime</i>	~10 MB
Locales (ICU)	~5 MB
Glue do Electron	~3 MB
Bundle do jogo (<code>app.asar</code>)	~80 KB

O “jogo em si” ocupa 0,11% do binário. Os outros 99,89% são o *runtime* necessário para hospedar uma página WebGL como aplicação *desktop*. Esta foi a escolha explícita do GDD; a alternativa seria usar o *webview* do sistema (Tauri ou Neutralino reduziriam o tamanho para 2 a 10 MB), mas o requisito de portabilidade absoluta e reprodutibilidade pesou na decisão por Electron.

7.5 Distribuição

O .exe é *não-assinado*, pois não há certificado de *code signing* de desenvolvedor Microsoft. Na primeira execução em Windows 10 ou 11, o *SmartScreen* exibe o aviso “O Windows protegeu seu PC”. A interação esperada do usuário é:

1. Clicar em *Mais informações*.
2. Clicar em *Executar mesmo assim*.

Após essa primeira execução, o *hash* do executável é registrado localmente e o aviso não reaparece. O modelo é coerente com a distribuição via *releases* do GitHub, o canal escolhido para entrega.

7.6 Repositório

O código-fonte e o .exe compilado estão disponíveis em:

<https://github.com/luig0c/Void-Descent>

A história de *commits* reflete a progressão real do projeto entre 03 de março e 09 de maio de 2026, com mensagens em formato *Conventional Commits*.

8. Backlog Técnico, Decisões e Aprendizados

Este capítulo documenta o caminho real do projeto: as decisões que tomamos, as ferramentas que escolhemos, o que ficou de fora e o que aprendemos no processo. Nem tudo era óbvio no GDD original; algumas escolhas só fizeram sentido ao bater em problemas concretos.

8.1 Decisões técnicas

8.1.1 JavaScript puro sem *framework*

Considerações descartadas: Phaser, PixiJS, Three.js. Optamos por JavaScript puro com WebGL direto pelos seguintes motivos:

- **Tamanho do bundle.** Phaser sozinho tem ~1 MB, e PixiJS tem ~500 KB. Nosso código fonte completo tem 77 KB. Um *framework* representaria mais de 90% do peso, contradizendo a proposta de procedural minimalista.
- **Aprendizado.** Implementar o *pipeline* de *shaders*, o *tinting* do atlas e a iluminação por vértice à mão deu domínio sobre o que cada parte faz.
- **Sem cerimônia de *boot*.** *Frameworks* têm *loaders*, cenas e sistema de eventos próprio. Para um jogo de tela única e fluxo linear, é *overkill*.

8.1.2 Atlas de tiles embarcado em *base64*

A `TILES_B64` em `game.js` é uma *string base64* de ~3 KB, que decodifica para um PNG 1024×16. Considerações descartadas:

- **PNG externo.** Adicionaria uma dependência de carregamento assíncrono e quebraria o *single-file deploy*. O jogo roda abrindo `index.html` sem servidor.
- **Geração 100% procedural via *shaders*.** Investigamos, mas o custo de tempo de implementação superava o benefício. O atlas pintado à mão é *tile-based* e mantém a estética *neon* desejada.

A escolha pelo atlas embarcado preserva a estética *neon* sem comprometer a *distribution story* de *single-page*.

8.1.3 Sonant-X em vez de Web Audio API direta

A Web Audio API permite síntese arbitrária via `OscillatorNode`, `BiquadFilterNode` e outros nós. Implementar oito SFX e uma faixa ambiente diretamente sobre essa API resultaria em centenas de linhas de *boilerplate*. O Sonant-X é uma biblioteca de ~200 linhas que abstrai oscilador, envelope, filtro, LFO e *delay* num único `instr` declarativo. Encurta o tempo de iteração das peças sonoras de horas para minutos.

8.1.4 Electron em vez de Tauri ou Neutralino

O GDD especifica Electron, e a implementação seguiu essa direção. Avaliamos alternativas durante o *packaging*:

Tecnologia	Tamanho .exe	Custo de migração
Electron 32 (escolhido)	73 MB	0
Tauri 2 (Rust webview)	~5–10 MB	Reescrita do <i>boot</i> , Rust toolchain
Neutralino	~2 MB	Reescrita do <i>boot</i> , depende de WebView2
Native C++ + OpenGL	<500 KB	Reescrita completa

A redução de Electron para Neutralino encolheria o **.exe** em ~ 35 vezes, mas o prazo próximo e o risco de *webview* ausente em máquinas antigas pesaram contra a migração.

8.1.5 *Spatial grid* 32×32 com células de 16

Decisão tomada após o modo de dificuldade alta elevar a contagem de inimigos para ~ 400 por *run*. O *loop* aninhado $O(n^2)$ original processava 80.000 pares por *frame*, o suficiente para causar quedas de 60 Hz para ~ 30 Hz em telas com muitos inimigos.

A escolha de células de 16 unidades é informada pelo tamanho da AABB da entidade (9 unidades). Cada entidade ocupa, no máximo, duas células, e a busca de vizinhos em 3×3 captura todas as colisões possíveis. Células menores aumentariam o *overhead* de *hashing*; maiores diluiriam o ganho.

8.1.6 *Lock-and-clear* via colisão lógica em vez de *tiles* físicos

A leitura natural do GDD, com a expressão “portas trancam”, sugeria adicionar *tiles* de parede dinamicamente nos corredores ao entrar em sala com inimigos. Descartado:

- O *vertex buffer* do WebGL é compilado uma vez no `load_level` e permanece estático durante o *gameplay*. Modificar *tiles* requer recompilar a geometria, custoso.
- Solução adotada: a função `entity_player_t.collides` é sobrescrita para checar se a posição alvo está dentro do retângulo da sala atualmente trancada. Sem custo de geometria, sem mudança visual.

A solução tem um *trade-off*: barreiras invisíveis. O efeito é mitigado com *feedback* sonoro e visual via `terminal_show_notice`.

8.2 Backlog: features descartadas ou postergadas

Tutorial explícito *Fora de escopo*. O GDD prescreve *teaching through play*. A primeira sala do Andar 1 ensina movimentação contra **Seekers** sem nenhuma tela de instrução, e a HUD é o único material didático.

Sistema de *achievements* *Descartado*. Adicionaria persistência entre *runs*, contradizendo o *permadeath* estrito e exigindo armazenamento local que o GDD evita.

Multiplayer cooperativo *Fora de escopo*. Não previsto no GDD.

Mais de três andares *Limitado pelo GDD*. A estrutura de três atos (apresentação, teste, clímax) é canônica no documento e foi mantida.

Validação *flood-fill* do mapa *Implementado parcialmente*. O GDD prevê uma fase de validação por *flood-fill* para garantir que todas as salas sejam alcançáveis. Na prática, o algoritmo de conexão sequencial entre salas ($i \rightarrow i+1$) já garante alcançabilidade por construção, e a validação se mostrou redundante. Documentado como TODO se geradores futuros forem mais sofisticados, como Voronoi.

Salvar e carregar *run* *Explicitamente fora*. *Permadeath* é decisão de design central; persistência quebraria o ciclo.

Inimigos novos no Andar 3 *Implementado*. Bombers entram apenas no Andar 3, conforme o GDD.

Boss com mais de duas fases *Considerado, descartado*. Duas fases é o ponto de equilíbrio observado em *playtesting*: pune complacência sem exaurir o jogador. Três fases prolongariam o confronto e diluiriam o impacto.

8.3 Ferramentas e *assets*

8.3.1 Estética visual

- **Atlas de tiles**: um único PNG 1024×16, embarcado como *base64* em `game.js`. Os 16 tiles cobrem chão, parede, inimigos, armas, explosões e ícones de UI. Foi o único *asset* pintado à mão.
- **Paletas por andar**: aplicadas via *hue shift* algorítmico (HSL) sobre o atlas no `regenerate_atlas`. Não há atlas separado por andar; todos derivam do mesmo PNG.
- **Sprites de inimigos**: 4 *frames* para `seeker` (animação) e sprite estático para `shooter`, `bomber` e `boss`.

8.3.2 Áudio

- **Sintetizador**: Sonant-X (*port* JavaScript de Sonant Live).
- **Faixa ambiente**: *drone* em Em, com 16 s de *loop* perfeito. Iterada cinco vezes até atingir uma textura aceitável; a primeira versão era *rock metal* agressivo, descartado por conflitar com a estética de terminal sci-fi.
- **SFX**: 8 *patches* (*shoot, hit, hurt, beep, pickup, terminal, explode, dash*).

8.3.3 Cadeia de *tools*

- **IDE**: VS Code.
- **Servidor de desenvolvimento**: `python3 -m http.server` no macOS, para servir `index.html` durante iteração.
- **Empacotamento**: `electron-builder 25` com *target portable*.
- **Documentação**: $\text{\LaTeX}$ (`latexmk`) e PlantUML para os diagramas UML. *Pipeline*: `.puml` → `.png` → `\includegraphics` → PDF final.
- **Versionamento**: Git e GitHub. Histórico de *commits* no formato *Conventional Commits*.

8.4 Aprendizados técnicos

8.4.1 *Race conditions* em `setTimeout` aninhados

A morte do jogador agenda `show_gameover_screen` em 1500 ms, e a morte do *boss* agenda `game_victory` em 2000 ms. Se ambas as mortes acontecem no mesmo *frame*, os dois `setTimeouts` ficam pendentes e podem disparar em qualquer ordem. Aprendizado: toda transição de estado precisa ser *idempotente* ou guardada por *state checks*; não confiar na ordem de execução de `setTimeout`.

8.4.2 Coordenadas do *mouse* requerem `getBoundingClientRect`

A primeira implementação usava `ev.clientX / canvas.clientWidth * canvas.width`, ignorando o *offset* do *canvas* no *viewport*. Funcionava quando o *canvas* ocupava 100% da janela, mas falhava quando havia margens (típico em *flexbox* centralizado). A correção via `canvas.getBoundingClientRect()` foi descoberta empiricamente, depois que o jogador precisava apontar para fora da arma para mirar.

8.4.3 `textAlign` do *canvas* 2D persiste entre chamadas

Bug clássico: setamos `textAlign = 'right'` para o POCAO e esquecemos de resetar para `'left'` antes de desenhar DASH: PRONTO, que renderizava fora da tela. Aprendizado: estados de contexto 2D (cor, alinhamento, fonte) são *sticky*; sempre setar explicitamente o que cada bloco precisa, ou usar `ctx.save()/restore()`.

8.4.4 *Lock-and-clear* precisa de *tracking* explícito por entidade

Versão ingênua: contar inimigos cuja posição está dentro do retângulo da sala. Falha: um *seeker* perseguindo o jogador pode sair temporariamente da sala pelo corredor, e a contagem cai para zero, liberando a sala falsamente. Solução: tagar cada inimigo no *spawn* com `e._room_idx = i` e contar por tag, não por posição. Aprendizado: para invariantes de *game state*, prefira *tags* persistentes a *queries* espaciais.

8.4.5 *Loop* de música ambiente exige alinhamento de *samples*

A primeira versão da música tinha um pequeno *gap* a cada *loop* (~0,8s) porque o `songLen` (em segundos) não era múltiplo exato do `rowLen` (em *samples*). Solução: escolher `rowLen` e o número de *patterns* de modo que $patterns \times 32 \times rowLen = songLen \times 44100$ exatamente. No nosso caso, $2 \times 32 \times 11025 = 16 \times 44100 = 705.600$ *samples*.

8.4.6 Cross-compile macOS para Windows funciona, com ressalvas

`electron-builder` cross-compila do macOS para Windows sem Wine *somente* se o *target* for *portable*. Alvos NSIS, MSI ou AppX exigem Wine ou execução em VM Windows. Aprendizado: a escolha do *target* é tão crítica quanto a do *framework* de empacotamento.

8.5 Mudanças de escopo em relação ao GDD

Durante o desenvolvimento, algumas especificações do GDD foram reinterpretadas, estendidas ou explicitamente divergiram. Esta seção lista cada caso, com a justificativa.

8.5.1 Divergências de *gameplay*

Velocidade dos Seekers

GDD: “*Seekers* são inimigos *melee* lentos cujo único comportamento é caminhar na direção do jogador em linha reta.”

Implementação: aceleração de 210 unidades por segundo, agressivos.

Razão: durante o *playtesting*, foi adotado um modo de dificuldade alta a pedido explícito da equipe. Manter *Seekers* lentos nesse modo tornaria o Andar 1 trivial. A versão original (lentos) seria adequada para um modo “normal”, mas não foi separada em uma *flag*.

Comportamento do *boss* na fase 2

GDD: “Na fase 2, ele abandona os projéteis, invoca **Seekers** como reforço para pressionar o jogador e avança diretamente em *melee*.”

Implementação: na fase 2, o *boss* mantém disparos em cone de três plasmas a cada 0,45s, avança em *melee* e invoca grupos de três **Seekers** a cada 1,8s.

Razão: o mesmo modo de dificuldade alta exigiu intensificar a fase 2. Sem projéteis, a fase 2 ficaria mais fácil que a 1, contra-intuitivo.

Densidade de inimigos por sala

GDD: não especifica número exato; prescreve aumento progressivo entre andares.

Implementação: 16 a 21 inimigos por sala no Andar 1, 23 a 28 no Andar 2 e 30 a 35 no Andar 3 (modo de dificuldade alta).

Razão: a contagem original (2 a 7 por sala) tornava o jogo trivial em *playtesting*; o aumento foi calibrado iterativamente.

Drops generosos no Andar 1, escassos no Andar 2

GDD: “Os drops nesse andar [Andar 1] são generosos. ... no Andar 2, a generosidade diminui de forma perceptível.”

Implementação: taxa uniforme de 50% de chance de poção por sala em todos os andares, com poção garantida na sala do baú.

Razão: simplificação do balanço. A diferenciação por andar adicionaria complexidade sem ganho proporcional dado o escopo do P2.

Sala do *boss* no Andar 3

GDD: “[Andar 3] possui apenas 3 salas regulares, incluindo uma sala de baú, seguidas pela arena final do *boss*.”

Implementação: 4 salas regulares mais a arena do *boss*, totalizando 5 salas no Andar 3.

Razão: ajuste empírico durante *playtesting*. Com 3 salas, o Andar 3 passava rápido demais e a expansão da dificuldade não tinha tempo de se sentir.

8.5.2 Divergências de *interface*

Tecla E para interagir

GDD: “E: Interagir (coletar item ou usar escada).”

Implementação: *auto-pickup* ao tocar; a escada também ativa ao toque.

Razão: decisão consciente de UX. Em *runs* curtas com muitos inimigos, a fricção de uma tecla extra para coletar prejudicaria o *flow state*. E ficou disponível como *slot* reservado para futuras ferramentas de interação, como portas opcionais ou NPCs.

Sistema de pontuação

GDD: $score = kills \times combo + itens \times 100$.

Implementação: $score += base_score \times combo$ a cada *kill* (incremental, não baseado em *kills* totais), com bônus de +100 por *pickup*.

Razão: a fórmula incremental dá *feedback* imediato no HUD a cada abate, em vez de calcular ao final. É semanticamente equivalente, pois *kills* altos com *combo* alto produzem *score* alto. A versão do GDD exigiria recálculo a cada *kill* ou ao final, sem benefício de UX.

Slot de passivo dedicado

GDD: “1 *slot de passivo* ... ocupado por modificadores que persistem durante toda a *run* (como velocidade aumentada, dano extra ou HP adicional).”

Implementação: o sistema de *shop* entre andares cumpre essa função. **REPARO**, **OVERCLOCK**, **AMPLIFIER**, **ACELERADOR** e **CADÊNCIA** são todos modificadores persistentes.

Razão: a estrutura é equivalente ao GDD em termos funcionais, mas empacotada como *shop* (escolha ativa entre opções) em vez de *drop* aleatório (escolha passiva). *Trade-off*: ganha agência do jogador, perde descoberta orgânica.

8.5.3 Divergências de geração

Validação por *flood-fill*

GDD: “Após essa etapa, uma validação por *flood-fill* percorre o mapa a partir do ponto de *spawn* do jogador para garantir que todas as salas sejam alcançáveis.”

Implementação: validação não implementada; alcançabilidade garantida por construção (o algoritmo conecta sala i a $i + 1$ sequencialmente).

Razão: redundância. A construção sequencial impede salas isoladas. Se o gerador for substituído por algoritmo mais sofisticado (Voronoi, BSP), a validação *flood-fill* se torna necessária.

Telegrafia de ataques

GDD: “Cada inimigo telegrafa seus ataques com um *flash* visual antes de efetivamente agir.”

Implementação: **Bomber** pulsa visualmente durante o *fuse* ($\sim 0,8s$); **Shooter** e **Seeker** não telegrafam.

Razão: **Seeker** é *melee* de contato, e a aproximação é a própria telegrafia. **Shooter** dispara projéteis visíveis em arco amplo, dando ao jogador 0,5 a 1,0s de janela de reação. Telegrafia adicional em ambos seria redundante e poluiria a HUD.

8.5.4 Adições não previstas no GDD

- **Modo de dificuldade alta:** levou a aumentar a contagem de inimigos, a velocidade e a agressividade do *boss*.
- **Bússola direcional na HUD:** pequeno triângulo apontando para o alvo mais próximo (inimigo, ou escada quando todos estão mortos), adicionado para mitigar a sensação de desorientação em mapas maiores.
- **Mute toggle (tecla M):** não previsto no GDD. Adicionado por boa prática de UX, evita interromper Discord, *streaming* ou apresentação acadêmica.
- **Indicador de sala atual na HUD (SALA X/Y):** adicionado durante a implementação do *lock-and-clear* para dar *feedback* de progresso dentro do andar.
- **Tela de pausa explícita:** o GDD lista ESC como pausa nos controles, mas não detalha a UI. Implementamos um *overlay* semi-transparente com o texto “PAUSADO”.

8.6 Cronograma: planejado vs executado

Marco	Planejado (GDD)	Executado
<i>Setup</i> inicial	Semana 1	03 a 12 mar
Sintetizador e RNG	Semana 1	12 mar
<i>Renderer</i> WebGL	Semana 2	22 mar
Geração de <i>dungeon</i>	Semana 2	01 abr
Sistema de entidades	Semana 3	20 abr
HUD e telas	Semana 4	22 a 30 abr
Polimento (<i>lock-and-clear</i> , <i>spatial grid</i>)	não planejado	02 a 05 maio
<i>Build</i> e empacotamento	Semana 4	07 a 09 maio
Documentação	não planejado	09 maio

O atraso na geração da *dungeon* (esperada na semana 2, executada no fim de março) refletiu a curva de aprendizado da geração procedural. Compensamos comprimindo o sistema de entidades (semana 3 oficial, executado em duas semanas e meia).

A fase de polimento (*spatial grid*, *lock-and-clear*, *melee*, poção, pausa, bússola, ajuste de HUD) não estava no GDD original. Emergiu de *playtesting* interno e da decisão de subir a dificuldade para o modo de dificuldade alta. Foi a fase com maior densidade de *commits*: cinco em quatro dias.

8.7 Uso de IA como ferramenta de apoio

Em pontos específicos do desenvolvimento, a equipe utilizou modelos de linguagem como ferramenta auxiliar, da mesma forma que se usa um *linter*, uma ferramenta de *static analysis* ou um colega para *rubber duck debugging*. As decisões de arquitetura, design, balanceamento e direção criativa foram tomadas pela equipe.

Os pontos onde a IA foi consultada são bem delimitados.

8.7.1 Abstrações matemáticas pontuais

- **Diferença angular normalizada** para o sistema de *melee*: a fórmula $\text{diff} = \text{atan2}(\sin(\alpha - \beta), \cos(\alpha - \beta))$ é o truque clássico para encontrar a menor diferença angular sem casos especiais em $\pm\pi$. A IA confirmou a derivação e ajudou a verificar o domínio do resultado.
- **Conversão $\text{HSL} \Rightarrow \text{RGB}$** para o *tinting* do atlas por andar: a equação envolve interpolação não trivial nos seis sextantes do cone HSL. A IA forneceu a versão canônica para verificação cruzada.
- **Dimensionamento do *spatial hash grid***: a justificativa para células de 16 unidades, baseada na relação entre o tamanho da AABB de 9 unidades e o custo de *hashing*, foi validada como argumento, não copiada.

8.7.2 Pesquisa de bibliotecas e *trade-offs*

- Comparativo entre Electron, Tauri, Neutralino e C++ nativo, considerando tempos de *build*, tamanho final do executável e custo de migração. A IA acelerou a coleta de dados que precisariam ser pesquisados em múltiplas fontes; a decisão de manter Electron foi da equipe, baseada no prazo e no requisito do GDD.
- Padrões consolidados de *melee combat* em *shooters* top-down (arco de *hitbox* temporário com *timer* curto) e de inventário de *slot* único em *roguelikes*, usados como referência durante o desenho da implementação.

- Estrutura de instrumentos do Sonant-X (parâmetros de envelope, filtro, LFO), consultada para entender qual combinação produz *drone* ambiente em vez de *drum* percussivo.

8.7.3 Revisão de código

A IA foi utilizada como segundo par de olhos para identificar *bugs* sutis que *linters* não capturam: *race conditions* entre `setTimeouts` (*gameover* contra *victory*), *state leaks* entre transições de tela (teclas *stuck*) e inconsistências de *textAlign* no *canvas* 2D que produziam elementos da HUD fora da tela. Cada *bug* apontado foi investigado, reproduzido e corrigido manualmente.

8.7.4 Polimento da documentação

Esta documentação L^AT_EX foi redigida pela equipe, com auxílio da IA para *boilerplate* de configuração de pacotes, ajuste fino de *layout* (helpers `\diagfig` e `\diagfiglandscape`) e revisão de estilo. O conteúdo técnico, com decisões de arquitetura, descrição de algoritmos, *trade-offs* e divergências em relação ao GDD, reflete diretamente as escolhas documentadas no código e nas conversas internas da equipe.

8.7.5 O que a IA não fez

Para que não haja ambiguidade: a IA não definiu o conceito do jogo, não desenhou o sistema de combate, não escolheu a estética visual, não escreveu o GDD original e não decidiu o ritmo dos andares. Esses são produtos do trabalho da equipe ao longo das treze semanas de desenvolvimento.

A IA foi usada onde foi mais útil: como acelerador de pesquisa, validador de matemática, revisor extra de código e assistente de redação técnica. Nunca como substituto do processo de design.

9. Considerações Finais

Void Descent começou com uma proposta ambiciosa: construir um *roguelike* top-down procedural completo em JavaScript puro, sem *frameworks*, sem *assets* externos e empacotado como executável *desktop*. Treze semanas depois, o resultado é um `.exe` de 73 MB que carrega 77 KB de código, gera mapas, áudio e comportamentos novos a cada *run*, e cumpre todos os critérios de aceite definidos no documento de requisitos.

9.1 O que foi entregue

O escopo executado contempla:

- Os oito módulos JavaScript da arquitetura, com responsabilidades isoladas e sem dependências circulares.
- Treze classes de entidade derivadas de `entity_t`, cobrindo jogador, quatro tipos de inimigos, *boss* de duas fases, projéteis, partículas e *pickups*.
- Nove estados de jogo (`menu`, `credits`, `quit`, `loading`, `playing`, `paused`, `shop`, `gameover` e `victory`), todos com transições idempotentes e *handlers* de entrada apropriados.
- Três andares com paletas distintas (ciano, amarelo, magenta), curva de dificuldade crescente e *boss* no Andar 3.
- *Lock-and-clear* de salas, *spatial grid* para colisão, sistema de *shop* com cinco *upgrades* persistentes, oito armas (quatro *ranged*, quatro *melee*) e *slot* de consumível.
- HUD funcional com vida em corações, andar, sala atual, score e combo, arma equipada e seu tipo, *dash*, poção, contagem de inimigos, bússola direcional e indicador de *mute*.
- Empacotamento Electron *portable* para Windows x64, com cross-compile do macOS, distribuído via GitHub.

9.2 O que aprendemos

A lição mais valiosa do projeto foi que a maior parte do tempo de desenvolvimento não está nas funcionalidades de fachada (telas, menus, HUD), mas nos detalhes invisíveis que separam um *prototype* de um produto. *Race conditions* de `setTimeout`, *stuck keys* entre transições, *textAlign* esquecido, inimigos escapando de salas trancadas. Cada um desses *bugs* parece pequeno isoladamente, mas o efeito acumulado é a diferença entre algo que “funciona” e algo que se sente bem de jogar.

A segunda lição foi sobre *trade-offs* de empacotamento. Empacotar uma página HTML como aplicação *desktop* via Electron custa 73 MB. Tauri ou Neutralino custariam menos, mas com risco de incompatibilidade em máquinas antigas. C++ nativo custaria menos ainda, mas exigiria reescrever tudo. Não há resposta universal: a escolha certa depende do prazo, do público alvo e dos requisitos do GDD. No nosso caso, manter Electron foi a decisão correta.

9.3 O que faríamos diferente

Em retrospecto, três pontos teriam reduzido o esforço total:

1. **Spatial grid desde o início**, não como refatoração tardia. O *loop* aninhado $O(n^2)$ funcionou enquanto havia poucos inimigos, mas o refator tardio tomou um dia inteiro de trabalho que poderia ter sido evitado.
2. **Pause e *lock-and-clear* planejados como parte da arquitetura**. Ambos foram adicionados depois e exigiram retoques em múltiplos arquivos. Tê-los desde o desenho inicial dos estados teria simplificado o código.
3. **Documentação progressiva**, em vez de concentrada no fim. Esta documentação foi escrita em uma única semana ao final do projeto, e isso exigiu reconstruir o raciocínio retrospectivamente para várias decisões. Uma página de *decision log* mantida ao longo do desenvolvimento teria capturado o porquê de cada escolha no momento em que foi feita.

9.4 O que vem depois

Há um conjunto claro de melhorias que ficariam para um eventual P3 ou para uma versão pós-disciplina:

- Modos de dificuldade selecionáveis no menu. O modo de dificuldade alta está *hardcoded* hoje; expor uma *flag* permitiria oferecer um modo “normal” alinhado com o GDD original.
- Telegrafia visual completa para **Seekers** e **Shooters**. Hoje apenas **Bombers** pulsam antes de explodir.
- Validação *flood-fill* do gerador de *dungeons*. Necessária se o algoritmo de placement for substituído por algo mais sofisticado.
- Migração para Neutralino, reduzindo o *.exe* para ~2MB. Exige apenas máquinas com WebView2, instalado por padrão em Windows 10 e 11 atualizados.
- Tabela de *high scores* local, mantida em *localStorage*. Reforçaria o *loop* macro de competição contra o próprio histórico, sem quebrar o *permadeath*.

9.5 Encerramento

O Void Descent existe porque quatro pessoas concordaram em entregar um jogo procedural completo em treze semanas, e seguraram esse compromisso até o fim. O documento de requisitos foi atendido; o GDD interno foi atendido com divergências documentadas; o código está versionado em repositório público; o executável compila e roda.

Mais importante, ao final do processo, o jogo é divertido de jogar. Esse era o critério verdadeiro, o que nenhum documento técnico consegue medir mas que define se o projeto valeu a pena. Nossa avaliação interna é que valeu.